

Unified Resilience Testing Framework for ShaktiDB / DistDB

SakthiDB (PostgreSQL) and DistDB

Document	Unified Resilience Testing Framework
Version	1.0
Date	28 July 2026
Owner	Database Reliability Engineering
Systems covered	SakthiDB / standalone PostgreSQL; DistDB distributed cluster
Scope	155 distinct scenarios across three tiers; 97 rated P0
Classification	CONFIDENTIAL — INTERNAL USE ONLY
Review cycle	Quarterly, or after any significant infrastructure or architecture change

Core philosophy

“Resilience is not about preventing failures; it is about designing systems that fail gracefully and recover quickly. If we have not tested a failure mode, we are not resilient to it — and if the test cannot fail, we have not tested it.”

- 1. Executive Summary
 - 1.1 Purpose
 - 1.2 The framework at a glance
 - 1.3 The durability principle
- 2. What Resilience Testing Is — Definition and Scope
 - 2.1 Working definition
 - 2.2 The five measurable properties
 - 2.3 Scope boundaries

3. Systems Under Test
 - 3.1 SakthiDB — HA cluster (Need to update based on Recommended architecture)
 - 3.2 DistDB — distributed cluster (Need to update HA Architecture)
 - 3.3 The key structural insight
 - 3.5 Support contract — guarantee versus precondition
 - 3.6 Environment Qualification Suite (EQS)
4. The Unified Three-Tier Model
 - 4.1 One implementation, many targets
 - 4.2 What the model requires of the harness
5. Coverage Matrix
6. SLA Targets and Measurement
 - 6.1 SLA tiers — targets are conditional on qualified infrastructure
 - 6.2 Metric definitions
 - 6.3 RTO must be decomposed
 - 6.4 RPO must be measured by transaction markers
 - 6.5 “First write” is not recovery
7. Scenario Specification and Execution Standard
 - 7.1 The scenario record
 - 7.2 Execution lifecycle
 - 7.3 Environment sensitivity controls the combinatorial explosion
 - 7.4 Fault injection without containerization
 - 7.5 The workload dimension is mandatory
8. Priority Method
9. Coverage Summary by Tier
10. Tier 1 — Node-Local Scenarios (NL)
 - 10.1 NL-D · Durability — the fsync-truth class
 - 10.2 NL-C · Crash and recovery
 - 10.3 NL-W · WAL and archiving
 - 10.4 NL-I · Integrity and corruption
 - 10.5 NL-R · Resource exhaustion
 - 10.6 NL-N · Noisy neighbour and contention
 - 10.7 NL-M · Maintenance, vacuum and wraparound
11. Tier 2 — Cluster Scenarios (CL)
 - 11.1 CL-F · Failover and promotion
 - 11.2 CL-S · Split-brain and fencing
 - 11.3 CL-R · Replication resilience
 - 11.4 CL-N · Network
 - 11.5 CL-C · Connection layer
 - 11.6 CL-B · Backup, restore and disaster recovery
 - 11.7 CL-X · Compound failures
12. Tier 3 — Distributed Scenarios (DX)
 - 12.1 DX-T · Two-phase commit and in-doubt transactions
 - 12.2 DX-M · Metadata and coordinators
 - 12.3 DX-S · Shard placement and rebalancing
 - 12.4 DX-Q · Distributed query execution

- 12.5 DX-P · Cross-node partition
- 12.6 DX-D · Multi-datacentre disaster recovery
- 13. Design Standards
 - 13.1 Durability cannot be validated by process kill
 - 13.2 Acceptance criteria must be falsifiable
 - 13.3 Failover suppression is a first-class requirement
 - 13.4 Noisy neighbour and grey failure are a distinct class
 - 13.5 Consistency checking must reach transaction-isolation anomalies
 - 13.6 One machine-readable source of truth
 - 13.7 Catalog validation — checks applied
- 14. Scope Boundaries and Deferred Items
- 16. Deliverables, Entry and Exit Criteria
 - 16.1 Deliverables
 - 16.2 Entry criteria
 - 16.3 Exit criteria
- 17. Risk Register
- 18. Constraints Already Settled
- 19. Open Decisions Required
- 20. Document Approval

1. Executive Summary

1.1 Purpose

This framework establishes a single, automated approach to validating the resilience of both our standalone PostgreSQL/SakthiDB deployments and our DistDB distributed cluster. One catalog, one harness and one set of measurement definitions serve both systems.

It exists to prove five things, by measurement rather than assertion:

- That acknowledged commits survive real failure — including storage failure and power loss, not merely process termination
- That service is restored automatically within a defined RTO
- That data integrity is preserved, and that corruption is always detected and never silent
- That the cluster does **not** fail over when it should not
- That every claim is backed by audit-ready, machine-verifiable evidence

1.2 The framework at a glance

Measure	Value	Meaning

Distinct scenarios	155	Authored once; reused across systems and node roles
P0 critical	97	Must pass — blocks release
P1 high	54	Must pass or carry documented, accepted remediation
P2 medium	4	Should pass — tracked
Reuse tiers	3	Node-local (NL) → cluster (CL) → distributed (DX)
Categories	20	Seven NL, seven CL, six DX
Systems covered	2	SakthiDB / standalone PostgreSQL, and DistDB
Apply to standalone	109	Tier 1 (54) + Tier 2 (55)
Apply to DistDB	155	All three tiers
Target RPO — Platinum and Gold	0	Conditional on qualified infrastructure (§6.1)

1.3 The durability principle

Read before approving any durability claim

A `kill -9` test cannot validate durability. Killing the postmaster terminates the process but leaves the operating system page cache fully intact — the kernel still flushes those dirty pages to disk afterwards. Such a test exercises PostgreSQL's WAL replay path and **nothing below it**.

A cluster configured with `fsync = off`, `synchronous_commit = off`, a write-back controller cache without battery backup, or a hypervisor in

cache=unsafe mode passes a process-kill test **every single time**, while losing committed data in a real power event.

Category **NL-D** (§10.1, six scenarios) exists to close exactly this gap. It is the highest-value work item in the programme and is sequenced first (§15). Until NL-D passes on a given storage class, **no RPO = 0 claim may be made for that storage class**.

2. What Resilience Testing Is — Definition and Scope

2.1 Working definition

Database resilience testing is the empirical validation that a database preserves its **durability** and **integrity** contracts and restores its **availability** and **performance** contracts within stated bounds, when subjected to the failure modes to which its deployment is actually exposed.

2.2 The five measurable properties

Every scenario in this framework measures at least one of the following. A test that measures none of them is not a resilience test and does not belong in this catalog.

Property	Question it answers	Primary metric
Durability	Do acknowledged commits survive?	RPO — transactions lost
Integrity	Is stored data still correct and self-consistent?	Corruption count, invariant violations
Availability	Does service return, automatically, in time?	RTO to first successful write
Performance recovery	Does service return to <i>useful</i> levels?	RTO-to-SLO
Detectability	Was the failure noticed and signalled?	MTTD, alert precision and recall

2.3 Scope boundaries

In scope

- Unplanned component failure
- Resource exhaustion
- Network fault
- Storage fault
- Data corruption
- Dependency failure

and the availability impact of *planned* change (rolling restart, upgrade, scaling) — because planned change is a leading source of unplanned outage.

Out of scope

SQL and functional correctness · pure throughput benchmarking (workload is a *substrate* here, not the object of study) · security and penetration testing · capacity planning · validating monitoring configuration as an end in itself.

Detection latency is measured *inside* scenarios as MTTD; standalone “does the alert fire” tests are not resilience tests.

3. Systems Under Test

3.1 SakthiDB — HA cluster (Need to update based on Recommended architecture)

A Patroni-managed PostgreSQL high-availability group: one primary, a synchronous replica and an asynchronous replica, with etcd as the consensus store, PgBouncer as the connection pooler, and pgBackRest for backup and point-in-time recovery.

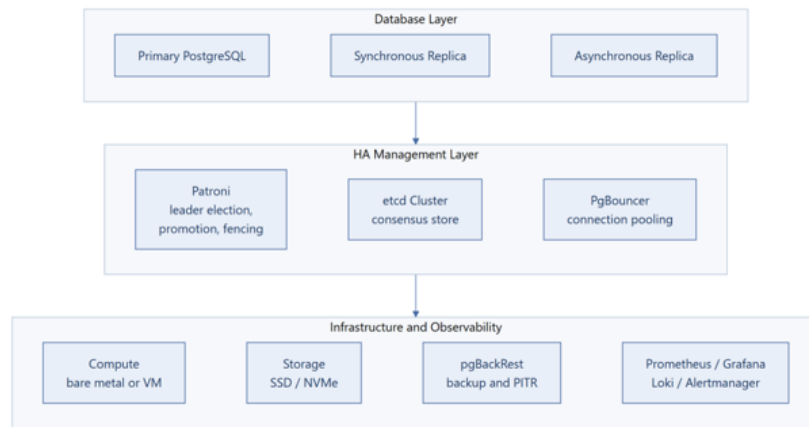


Figure 1 — SakthiDB / standalone PostgreSQL HA deployment.

3.2 DistDB — distributed cluster (Need to update HA Architecture)

Distributed database: a Primary Coordinator holding global metadata, horizontally scaled stateless Query Coordinators, and sharded worker clusters, with an optional cross-datacenter disaster-recovery site.

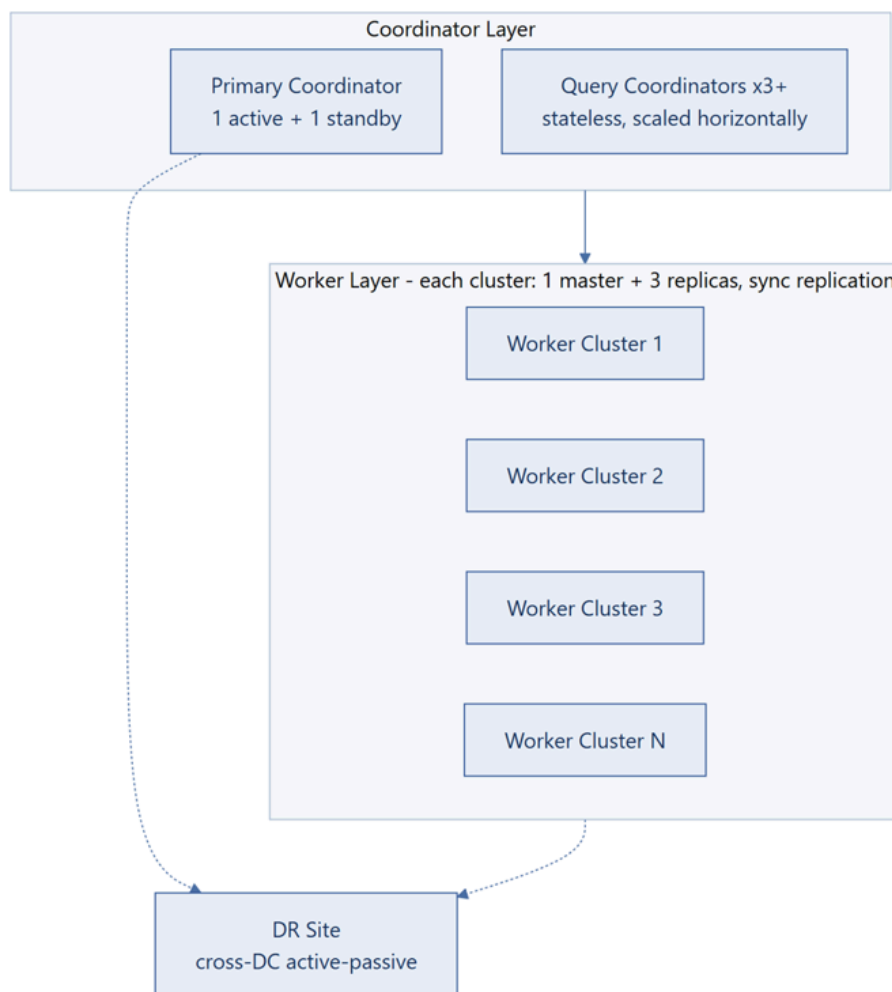


Figure 2 — DistDB cluster topology

The architectural properties below are confirmed against the DistDB design document and drive the whole of Tier 3.

Property	Behavior
Two-phase commit	Implemented — mandatory for DDL and metadata, optional for DML
Worker topology	4-node clusters (1 master + 3 replicas), synchronous replication, automatic promotion
Consistency model	Strong consistency via 2PC when enabled; per-shard atomicity when disabled
Transaction tracking	64-bit GXID
Primary Coordinator	Exactly one; holds global metadata, the exclusive metadata lock, and the 2PC transaction manager
Query Coordinators	Stateless; local metadata cache invalidated by the Primary Coordinator
Reference tables	Replicated to all workers via 2PC
Consensus	No Raft or quorum protocol for metadata — a strict single-primary policy

3.3 The key structural insight

DistDB is ShaktiDB-derived. **Its Primary Coordinator, its Query Coordinators and every worker node are ShaktiDB instances.**

Decomposing this work by product would therefore duplicate every node-level **ShaktiDB** failure mode across two catalogs, where the copies would promptly drift apart. The correct decomposition is by **scope of failure**, not by product. That is the three-tier model in section 4.

These databases ship as a **product**. Customers deploy them on infrastructure we do not own, choose or observe — dedicated and shared alike, and both must be supported. The environment is therefore a **first-class test axis**, not a fixed assumption.

The constraint that shapes everything below

On infrastructure we do not own, we cannot inject a noisy neighbour — we do not control the neighbour. Category NL-N can therefore only be *executed* in our own lab, across representative environment classes. What ships to the customer is not the injection; it is **detection, graceful degradation, honest SLA tiering, and a qualification tool** they can run against their own infrastructure.

The same applies to power loss: we can force-stop our lab VMs, but we cannot cut power on a customer’s managed cloud instance.

Class	Compute	Storage	Co-tenancy	Power-loss injectable ?	Noisy neighbour injectable ?
E1	Dedicated bare metal	Local NVMe / SSD, BBU RAID	None	Yes — physical cut plus IPMI. Strongest evidence.	External: N/A by design. Internal still applies — NL-N-06 (backup) and NL-N-07 (analytics) contend on any host
E2	VM on a dedicated host	Local or SAN	None (host dedicated)	Yes — hypervisor force-off	Yes — from co-located

					VMs we control
E3	Shared / multi-tenant cloud VM	Network block (EBS-class)	Yes, and invisible	Cloud force-stop — weaker evidence	No — the neighbour is not ours
E4	Burstable / credit-based instance	Network block plus burst credits	Yes, plus credit cliffs	Cloud force-stop	Partially — credit depletion is self-inflictible
E5	Containerized / Kubernetes	—	—	Not applicable — the database is not containerized. Out of scope until it is.	—

Storage class is the independent variable for durability

Durability is determined by the *storage* stack, not the compute class; availability is determined by *compute and network* predictability. These are independent preconditions and should be qualified separately. A customer on dedicated bare metal with a lying RAID controller has worse durability than one on a shared cloud VM with a correctly-flushing network volume.

Storage classes to qualify: local NVMe · local SSD behind BBU RAID · SAN · network block (EBS-class) · NFS or other network filesystem.

3.5 Support contract — guarantee versus precondition

Because the customer chooses the infrastructure, several SLA commitments are **conditional**. Stating precisely which, and shipping the tool that measures the condition, is the core product obligation this framework serves.

Property	We guarantee unconditionally	Only if customer infrastructure provides	Verified by
RPO = 0	Committed transactions survive process crash, node crash and failover, given honest storage	Storage honours fsync/FUA · no volatile write cache without BBU · fsync=on and synchronous_commit=on not overridden · sync standby healthy	NL-D-01 ... NL-D-06 → EQS-D
RTO	Automatic detection, election and promotion within the budget for the qualified tier	CPU steal below threshold · storage p99 fsync below threshold · network RTT below threshold to peers and to the consensus store	CL-F-08 → EQS-P
No split-brain	Correct fencing and self-demotion in every partition scenario tested	Consensus store on an independent failure domain · clock skew within bound	CL-S-01 ... CL-S-06

Integrity	Corruption is detected and surfaced, never silent	data_checksums =on at initdb time	NL-I-01 ... NL-I-09
Degraded-mode behaviour	Fully ours. Under contention we detect, declare degradation, and do not fail over spuriously — on any infrastructure	— nothing; this is unconditional	NL-N-01 ... NL-N-08

The asymmetry is deliberate and worth stating to customers. We cannot promise *performance* on infrastructure we do not control — but we can and do promise *correct behaviour under degradation*: no data loss beyond the qualified RPO, no spurious failover, no silent corruption, and an honest declaration of degraded state. Category NL-N is what discharges that promise, which is why it is unconditional in the table above.

3.6 Environment Qualification Suite (EQS)

A packaged subset of this framework, runnable by a Support engineer or by the customer against *their* infrastructure before go-live. It converts “which SLA tier does this deployment qualify for?” from a sales conversation into a measurement.

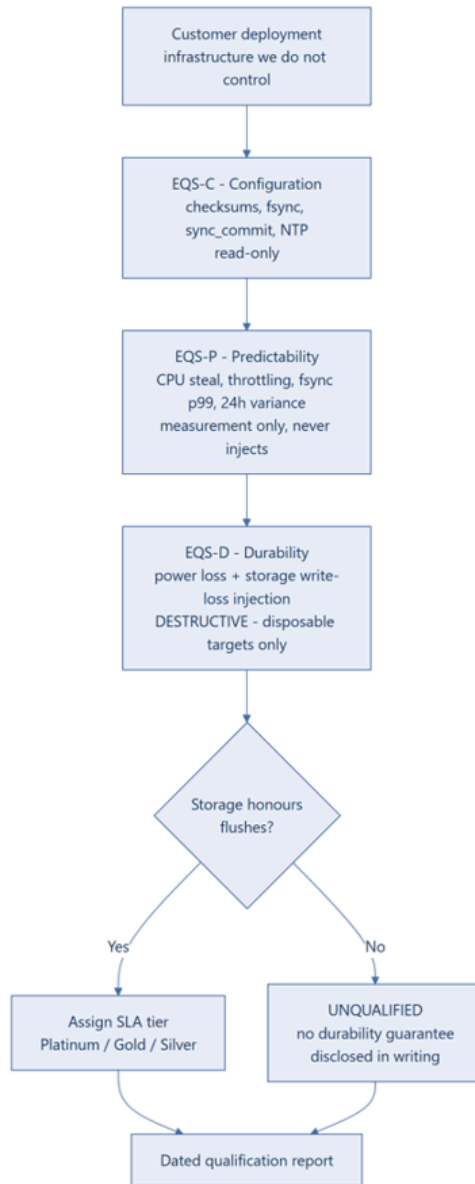


Figure 3 — Environment Qualification Suite: gating flow from deployment to assigned SLA tier.

Suite	Derived from	Measures	Gates
EQS-D — Durability	NL-D-01, 02, 03, 05, 06	fsync honesty, power-loss survival, torn-page handling, write-cache volatility	RPO = 0 eligibility. Fail ⇒ no durability guarantee may be offered
EQS-P — Predictability	NL-N-01, 02, 03, 08	CPU steal %, cgroup throttling,	RTO tier assignment

	(measurement-only variants)	fsync p99, sustained variance over 24 h	
EQS-C — Configuration	NL-I-09 plus configuration assertions	checksums, fsync, synchronous_commit, NTP, consensus-store placement	Any guarantee at all

Output

A dated qualification report naming the SLA tier the environment supports, the measured values behind it, and any disclosed limitations. It is re-run on any infrastructure change — instance family, storage class, hypervisor, or host move. This is a *product deliverable*, not an internal test artefact.

Note that EQS-P is **measurement-only**: it observes the steal, throttling and variance that the customer's real neighbours produce. It does not inject anything, because on shared infrastructure we cannot.

4. The Unified Three-Tier Model

Scenarios are organised by **scope of failure**: what breaks, not which product it broke in. A node-local fault is a node-local fault whether the node is a standalone primary or a DistDB worker replica.

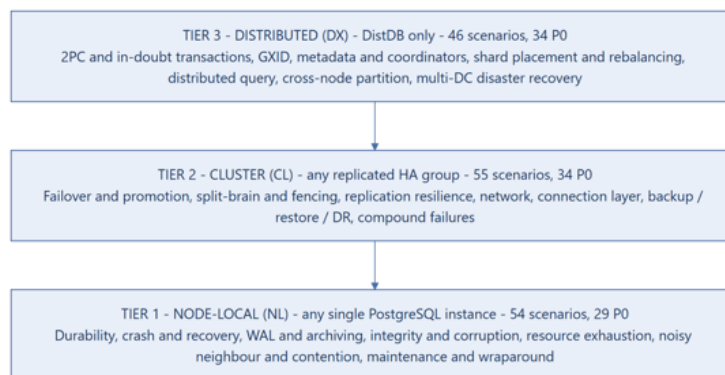


Figure 4 — The three tiers. Each tier adds scope; each inherits everything below it.

- **Tier 1 — NL, node-local.** Applies to any single PostgreSQL instance. 54 scenarios, 29 of them P0.
- **Tier 2 — CL, cluster.** Applies to any replicated HA group. 55 scenarios, 34 of them P0.
- **Tier 3 — DX, distributed.** DistDB only. 46 scenarios, 34 of them P0.

4.1 One implementation, many targets

A Tier-1 scenario is authored once. The harness's target resolver then executes it against the standalone instance, against the DistDB Primary Coordinator, against each Query Coordinator, and against every worker node. The *test code* does not change — only the target selector.

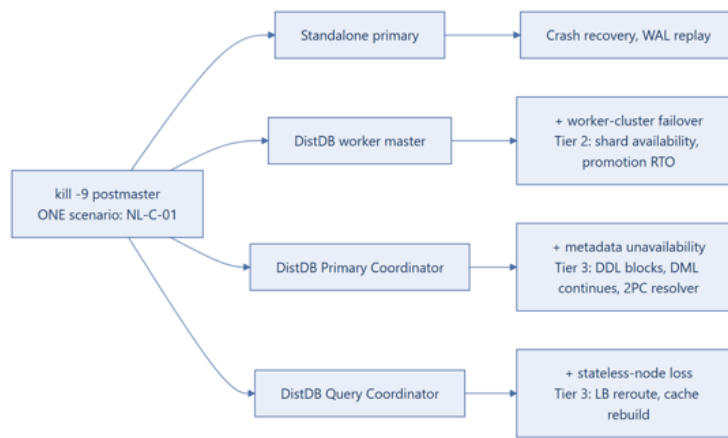


Figure 5 — Topological amplification: one authored scenario, four targets, four assertion sets.

kill -9 postmaster is **one** scenario, NL-C-01:

Applied to	Exercises	Additional assertion layered on
Standalone primary	Crash recovery, WAL replay	—
DistDB worker master	Crash recovery <i>and</i> worker-cluster failover	Tier 2: shard availability, promotion RTO
DistDB Primary Coordinator	Crash recovery <i>and</i> metadata unavailability	Tier 3: DDL blocks, DML continues, 2PC resolver
DistDB Query Coordinator	Crash recovery <i>and</i> stateless-node loss	Tier 3: load-balancer reroute, cache rebuild

Same fault, same code, four different assertion sets — rather than four separately authored scenarios that drift apart over time.

4.2 What the model requires of the harness

This model imposes exactly one requirement on the harness: **a target resolver that can enumerate PostgreSQL/Shakti instances by role**. Given applies_to: [distdb.worker], it must expand to every worker master and replica and run the scenario against each. Resolver strategies are needed for Patroni, for label selectors, for direct host lists, and for DistDB node roles.

5. Coverage Matrix

System	Tier 1 (NL)	Tier 2 (CL)	Tier 3 (DX)	Distinct scenarios	Execution instances
Standalone PostgreSQL / SakthiDB	✓ 54	✓ 55 (if HA)	—	109	~109
DistDB	✓ 54 × 4 roles	✓ 55 × N clusters	✓ 46	155	~295+

155 scenarios cover both systems. Because the decomposition is by scope of failure rather than by product, there is no cross-catalog duplication to maintain, and every node-level failure mode is exercised against every node role.

“Execution instances” counts *runs*, not scenarios to author and maintain.

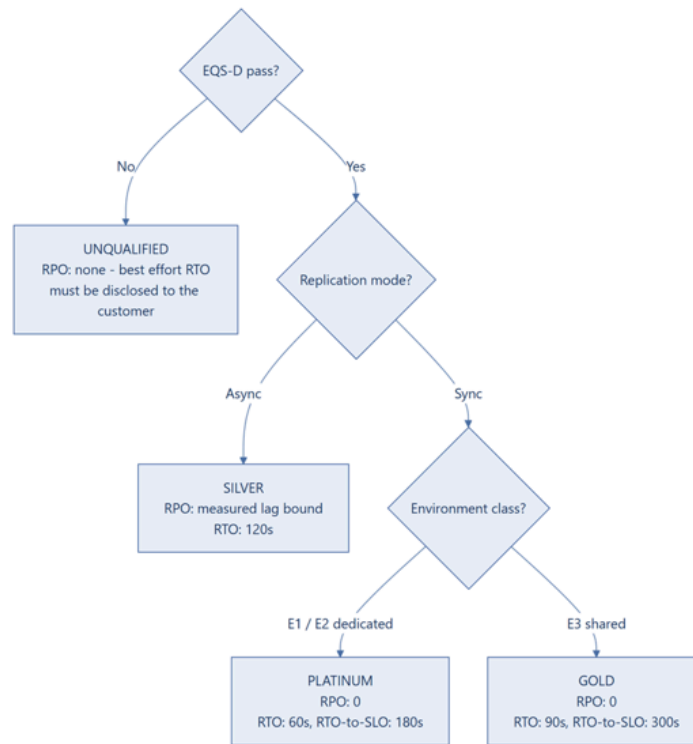
Tier	Categories	Scenarios	P0	P1	P2
TIER 1 — NL	Durability · Crash · WAL · Integrity ·	54	29	23	2

	Resources · Noisy Neighbour · Maintenance					
TIER 2 — CL	Failover · Split-brain · Replication · Network · Connection · Backup/DR · Compound	55	34	19	2	
TIER 3 — DX	2PC · Metadata · Shards · Query · Partition · Multi-DC	46	34	12	0	
TOTAL	20 categories	155	97	54	4	

6. SLA Targets and Measurement

6.1 SLA tiers — targets are conditional on qualified infrastructure

A single unconditional SLA is not honest for a product deployed on customer-chosen infrastructure. Targets are therefore **tiered by qualified environment class**, with the tier assigned by the Environment Qualification Suite (§3.6) rather than asserted in a contract.



Tier	Qualifying environment	RPO	RTO	RTO-to-SLO	Notes
Platinum	E1 / E2 · EQS-D pass · sync replication	0	≤ 60 s	≤ 180 s	Full guarantee. Dedicated infrastructure, honest storage.
Gold	E3 general purpose · EQS-D pass · sync replication	0	≤ 90 s	≤ 300 s	RPO fully preserved; RTO relaxed to absorb shared-compute variance.
Silver	E3 / E4 · EQS-D	≤ measured	≤ 120 s	Best effort	RPO > 0 <i>by design</i>

	pass · async replication	lag bound			<i>choice,</i> not by infrastruct ure defect. The bound must be measured and disclosed.
Unqualifie d	EQS-D fail — storage does not honour flushes	None	Best effort	—	No durability guarantee may be offered. The failure and its cause must be disclosed to the customer in writing.

The **Unqualified** row is the one that matters commercially. If a customer deploys on storage that discards flushes, no amount of correct database behaviour can deliver $RPO = 0$ — and a framework that reports “pass” in that situation is actively harmful. EQS-D exists to make that condition visible *before* go-live rather than after the first power event.

6.2 Metric definitions

Metric	Standalone / SakthiDB	DistDB	Definition

RPO	Per tier	Per tier	Committed transactions lost, measured by transaction markers
RTO (first write)	Per tier	Per tier, per shard	Fault injection → first successful write
RTO-to-SLO	Per tier	Per tier	Sustained $\geq 80\%$ baseline TPS at $p99 \leq 1.5\times$ baseline, held 60 s
Failover time	≤ 30 s (Platinum)	≤ 30 s worker / ≤ 60 s PC	Leader change complete and accepting writes
MTTD	≤ 10 s	≤ 10 s	Fault → detection signalled. Unconditional — detection is ours.
Data integrity	100%	100%	Zero corruption, zero invariant violations. Unconditional given checksums enabled.
Spurious failover rate	0	0	Unconditional — must hold on every environment class, including E3 and E4

DR RTO / RPO	Per DR policy	Per DR policy	Measured at production data volume, never assumed
---------------------	---------------	---------------	---

6.3 RTO must be decomposed

Measuring RTO as a single scalar hides where the budget is spent and makes tuning blind. When RTO regresses from 22 s to 48 s, one number tells you nothing.

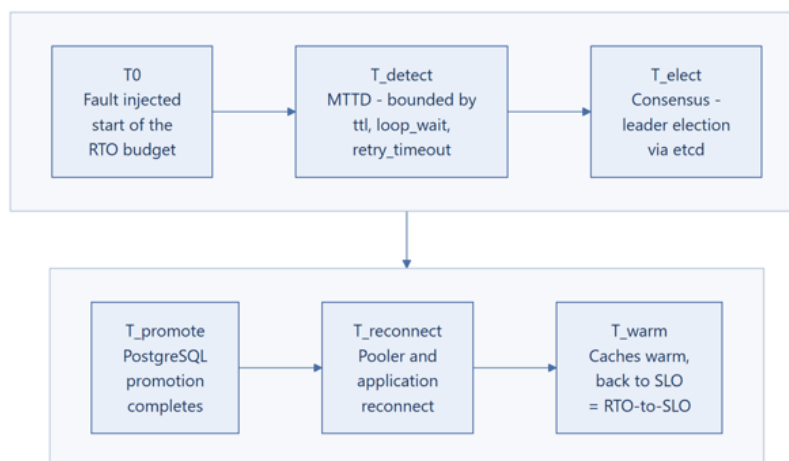


Figure 7 — The five components of RTO. Each is measured and budgeted separately.

$$\text{RTO} = \text{T_detect} + \text{T_elect} + \text{T_promote} + \text{T_reconnect} + \text{T_warm}$$

(MTTD) (consensus) (PG promote) (pooler/app) (to SLO)

For a Patroni cluster, T_detect is bounded by ttl, loop_wait and retry_timeout.

This is usually where most of the RTO budget actually goes -- and it is tunable.

Every Tier-2 failover scenario records all five components separately, and each component has its own budget.

6.4 RPO must be measured by transaction markers

This is the only reliable method. Before each commit the client appends a marker to a **local journal file** and flushes it; that file — not the database — is the source of truth for what the

client believed was committed. After recovery, the two sets are differenced.

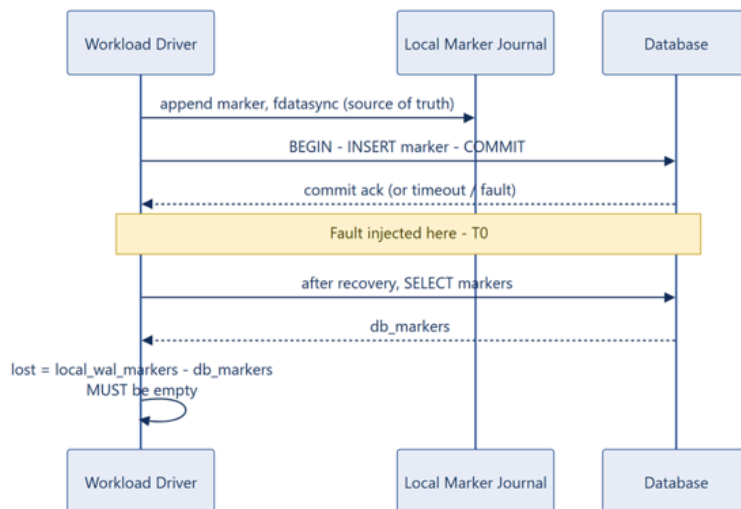


Figure 8 — Transaction-marker protocol for measuring RPO.

```
lost_transactions = local_wal_markers - database_markers # MUST be empty
```

Any scenario measuring RPO must declare `transaction_markers: true` in its workload block.

6.5 “First write” is not recovery

A freshly promoted replica has a cold shared buffer cache and a cold OS page cache. It will accept a single write within seconds while serving production traffic at a fraction of baseline throughput for several minutes. Reporting RTO-to-first-write alone materially overstates recovery. **Both figures are reported for every failover scenario.**

7. Scenario Specification and Execution Standard

Every scenario is specified with all of the fields below. A scenario missing any field is not ready to execute. This record is the machine-readable source of truth from which the catalog documents are generated.

7.1 The scenario record

```
id: NL-D-02
name: Durability under host power loss
tier: node_local
```

applies_to: [standalone, distdb.pc, distdb.qc, distdb.worker]

steady_state: # measured BEFORE injection, must hold to
proceed

tps_min: 800

p99_latency_ms_max: 50

replication_lag_s_max: 1

duration_s: 120

workload:

profile: oltp_write_heavy # idle | oltp_read | oltp_write_heavy |

concurrency: 64 # bulk_load | long_analytic | mixed

transaction_markers: true # required for any RPO-measuring scenario

fault:

type: host_power_loss

method: virsh destroy # NOT kill -9 -- see section 13.1

target: primary

duration: permanent

blast_radius:

max_nodes_affected: 1

environment: [lab, staging] # never production

abort_if:

- replication_lag_s > 300
- secondary_node_unhealthy

---- COMPOUND SCENARIOS ONLY (category CL-X) -----

```
# A second fault, armed on an OBSERVED STATE rather than a timer. "20
seconds

# in" and "the instant promotion begins" are different tests; only the
second

# reliably hits the fragile window.

compound:

  fault_b:

    trigger: on_state      # on_state | on_metric | after_phase

    condition: patroni_state == "promoting"

    timeout_s: 120        # if the state never arrives, abort -- do not
                          # silently degrade to a single-fault test

    type: process_kill

    target: sync_standby

  blast_radius_override:

    max_nodes_affected: 2  # documented exception; 1 everywhere else

# -----
-

measure: [rpo_txn, rto_first_write_s, rto_to_slo_s, mttd_s,
          corruption_count, t_detect_s, t_elect_s, t_promote_s]

accept:      # every predicate numeric or boolean

  • rpo_txn == 0

  • rto_first_write_s <= 60

  • rto_to_slo_s <= 180

  • corruption_count == 0

  • amcheck_errors == 0

environment:      # the deployment axis
```

```

classes: [E1, E2]      # where this scenario can actually be executed

env_sensitive: true     # does the RESULT depend on environment
class?

                        # true → must run on every supported class
                        # false → run once on the reference environment

priority: P0

```

7.2 Execution lifecycle

Every run follows the same eight-phase machine. Phases are bounded by timeouts; an unbounded wait is a defect.

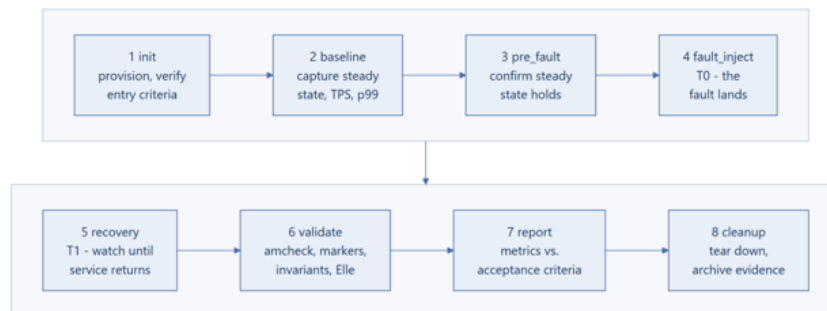


Figure 9 — The eight-phase test lifecycle. T0 is fault injection; T1 is the start of recovery measurement.

Safety is enforced by the harness, not by convention

Every scenario declares a blast radius and abort conditions. The Safety Controller enforces a kill switch, a circuit breaker, a maximum-affected-nodes cap, and a timeout on every operation. The cap is **one node** for every scenario except the compound category (§11.7), where it is two.

7.3 Environment sensitivity controls the combinatorial explosion

155 scenarios × 4 environment classes would be 620 runs, which is unmanageable and mostly redundant. Most scenarios are **environment-insensitive**: a CLOG corruption test behaves identically on bare metal and on a shared cloud VM, so it runs once on the reference environment.

Only **environment-sensitive** scenarios multiply — those whose *result*, not merely their execution, depends on the infrastructure underneath:

Category	Sensitive?	
NL-D Durability	Yes — all	The entire point is measuring the storage stack. Must run on every storage class offered.
NL-N Noisy neighbour	Yes — all	Contention behaviour is the property under test.
CL-F Failover	Partly	The RTO budget shifts with compute and network variance. CL-F-01/02/03/08 are sensitive; the rest are not.
CL-B Backup and restore	Partly	CL-B-05 (restore RTO) is throughput-bound and therefore sensitive.
NL-C, NL-W, NL-I, NL-M	No	PostgreSQL-internal behaviour. Reference environment only.
CL-S, CL-R, CL-C, CL-N	No	Protocol and configuration behaviour, not infrastructure behaviour.
DX-* (all)	No, except DX-S-08 and DX-Q-07	Distributed protocol behaviour is infrastructure-independent; the two

		co-tenancy scenarios are not.
--	--	-------------------------------

Result

21 environment-sensitive scenarios (NL-D 6 + NL-N 8 + CL-F-01/02/03/08 + CL-B-05 + DX-S-08 + DX-Q-07) × 4 classes = 84 runs, plus the remaining 134 run once = **218 runs** rather than 620 — and the multiplied set is exactly the set a customer cares about when asking “will this work on *my* infrastructure?”

7.4 Fault injection without containerization

The database is not containerized

Chaos Mesh and LitmusChaos are Kubernetes-native: they inject faults via custom resources against pods. Neither is usable against our deployment model today, and neither can be the primary injection path.

One important qualification: the Chaos Mesh project also ships `chaosd`, a standalone agent and CLI for **physical machines and VMs** — process kill, network, disk, stress, clock and JVM faults, with no Kubernetes involved. `chaosd` is directly applicable to E1–E4 and is a credible alternative to hand-rolling the injection layer. LitmusChaos has no equivalent and is genuinely Kubernetes-only.

Tool	Usable today?	Notes
Chaos Mesh (Kubernetes controller and CRDs)	No	Requires Kubernetes; a CRD-builder adapter is deferred until E5 exists
` <b>chaosd</b> ` (Chaos Mesh, standalone)	Yes	Physical and VM agent. Covers most of the table below behind one interface — evaluate before building bespoke adapters

LitmusChaos	No	Kubernetes-only, with no standalone mode. Deferred indefinitely.
Direct OS tooling (SSH, systemd, tc, dm-*, libvirt)	Yes	Always available; the fallback and the reference implementation

Fault class	Non-containerized — use today	Containerized — future, if and when E5 arrives
Process kill	SSH plus kill -9 / systemctl kill -s SIGKILL	Chaos Mesh PodChaos
CPU / memory limits	systemctl set-property CPUQuota=, MemoryMax=, IOReadBandwidthMax =	Chaos Mesh StressChaos, Kubernetes limits
Network	tc netem, iptables / nftables	Chaos Mesh NetworkChaos
Storage / I/O	dm-flakey, dm-delay, dm-error, fio contention, blkio cgroup	Chaos Mesh IOChaos
Node / power	virsh destroy, IPMI power-off, cloud force-stop API	Node drain or eviction
Clock	chronyc, date, libfaketime	Chaos Mesh TimeChaos

cgroups do not require containers

systemd places every service in a cgroup by default on any modern Linux distribution, so `systemctl set-property postgresql.service CPUQuota=50%` reproduces CFS throttling on a plain VM. NL-N-02 is therefore fully testable today — throttling simply becomes *more* likely later, if the product is containerized.

Recommendation: build the SSH/systemd/tc/dm-* injection path as primary, behind a FaultInjector adapter interface, so that container-native injectors can plug in unchanged if containerization lands later.

7.5 The workload dimension is mandatory

The same fault produces entirely different outcomes against an idle database, a 5,000 TPS OLTP workload, a long-running analytic query, and a bulk COPY. Workload is therefore a declared parameter of every scenario — never chosen ad hoc at execution time.

8. Priority Method

A four-factor weighted score, applied uniformly to both systems and to all three tiers.

Factor	Weight	High (3)	Medium (2)	Low (1)
Business Impact	40%	Data loss, complete outage, SLA breach	Partial outage, degraded performance	Minor disruption, no data impact
Likelihood	25%	Common failure mode (hardware, network)	Occasional occurrence	Rare edge case
Detection Difficulty	20%	Silent failure, delayed detection	Requires cross-system correlation	Immediate and obvious
Recovery Complexity	15%	Manual intervention,	Automated but complex	Simple automatic

		data recovery		recovery
--	--	------------------	--	----------

Priority	Score	Count	Gate
P0 Critical	2.5 – 3.0	97	Must pass — blocks release
P1 High	1.8 – 2.4	54	Must pass, or carry documented remediation
P2 Medium	1.0 – 1.7	4	Should pass — tracked

9. Coverage Summary by Tier

Section 5 gave the totals. This section summarizes coverage one level down — category by category, with a one-line description of what each actually tests — so the detail in §10–12 can be read against a map rather than discovered cold.

Category	What it tests	Scenarios	P0
TIER 1 — NODE-LOCAL · 54 scenarios, 29 P0 · any single PostgreSQL instance			
Durability	Storage and power-loss durability — the fsync-truth class	6	5
Crash & Recovery	Process termination, checkpoint, and	7	4

	large-transaction crashes		
WAL & Archiving	WAL segment integrity and archive continuity	7	4
Integrity & Corruption	Detection of heap, index, CLOG and control-file corruption	9	6
Resource Exhaustion	Memory, disk, connection and file-descriptor limits	9	4
Noisy Neighbour	CPU steal, cgroup throttling, burst-credit depletion, grey failure	8	4
Maintenance & Wraparound	Vacuum, autovacuum starvation, XID and multixact wraparound	8	2
TIER 2 — CLUSTER · 55 scenarios, 34 PO · any replicated HA group			
Failover & Promotion	Automatic recovery within RTO, including	10	7

	failover suppression		
Split-Brain & Fencing	No two nodes ever accept writes at once	6	4
Replication	Sync and async replication, quorum degradation, slot bloat	10	6
Network	Latency, packet loss and partition at the cluster level	6	1
Connection Layer	Pooler behaviour and reconnect storms during failover	5	2
Backup, Restore & DR	Backup integrity, PITR accuracy, restore at production volume	9	5
Compound Failures	Two concurrent faults — the second injected when the system is most fragile	9	9
TIER 3 — DISTRIBUTED · 46 scenarios, 34 PO · DistDB only			

Two-Phase Commit	In-doubt transactions, coordinator failure mid-commit	11	8	
Metadata & Coordinators	Primary and Query Coordinator failure, metadata consistency	9	7	
Shard Placement	Rebalancing, node loss, reference-table replication	8	6	
Distributed Query	Partial results, stragglers, repartition failures	7	4	
Cross-Node Partition	Network splits inside the cluster, convergence on healing	6	6	
Multi-DC Disaster Recovery	Cross-datacentre failover, DR RTO and RPO	5	3	
TOTAL — 20 categories		155	97	

Standalone PostgreSQL / SakthiDB runs Tiers 1 and 2 (109 scenarios). DistDB runs all three tiers (155), with every Tier-1 category above re-executed against the Primary Coordinator, each Query Coordinator, and every worker role (§4.1).

10. Tier 1 — Node-Local Scenarios (NL)

Applies to every single PostgreSQL instance — standalone SakthiDB, the DistDB Primary Coordinator, every DistDB Query Coordinator, and every DistDB worker master and replica. Authored once, executed against all.

10.1 NL-D · Durability — the fsync-truth class

No RPO = 0 claim is valid until this category passes. These are the only tests in the framework that can detect a dishonest storage stack.

ID	Scenario	Fault injection	Acceptance criterion	Pri
NL-D-01	fsync honesty baseline <i>(screening diagnostic)</i>	pg_test_fsync plus diskchecker against the data and WAL volumes	Baseline recorded per volume and carried into the NL-D-02/03 report; anomalous flush rates flagged as <i>suspected</i> volatile cache. Cannot by itself prove or disprove honesty — a non-volatile cache legitimately acknowledges fast. Gating is by NL-D-02/03.	P1

NL-D-02	Host power loss under write load	VM force-off / virsh destroy at 1000 TPS	rpo_txn == 0; corruption_count == 0; the database starts unattended	P0
NL-D-03	Storage write-loss injection	dm-flakey drops writes for 5 s during a commit burst	rpo_txn == 0 OR startup refuses with an explicit error — never silent loss	P0
NL-D-04	Durability configuration matrix	synchronous _commit ∈ {off, local, remote_write, on, remote_apply} × power loss. <i>off and local apply to any instance; the remaining three require a sync standby and are evaluated on clustered targets — see CL-R-04.</i>	Measured RPO matches the documented contract for each applicable level, exactly. For off, loss is bounded by 3 × `wal_writer_delay` , not merely “some loss”	P0
NL-D-05	Torn page / partial write	Power loss mid-checkpoint	FPW on: corruption_count == 0.	P0

		with full_page_writes on, then off	FPW off: the torn page is <i>detected</i> and not silent — note this detection depends on data_checksums being enabled; without checksums a torn page is silent, which is itself the finding	
NL-D-06	Volatile controller cache detection	Power loss with write-back cache enabled and no BBU	The test detects and reports the durability violation	P0

10.2 NL-C · Crash and recovery

ID	Scenario	Fault injection	Acceptance criterion	Pri
NL-C-01	Immediate process termination	kill -9 postmaster under load	rpo_txn == 0; rto_first_write_s ≤ 60; amcheck_errors == 0	P0
NL-C-02	Crash during checkpoint	kill -9 while the checkpointer is active	corruption_count == 0; recovery completes unattended	P0

NL-C-03	Crash during a large transaction	kill -9 during a 10M-row INSERT	Transaction fully absent; no partial rows; FK invariants hold; Elle reports no G0/G1a/G1b/G1c/G-single/G2 anomalies in the recorded history at the declared isolation level	P0
NL-C-04	Recovery time versus WAL volume	Crash at 1 / 4 / 16 GB of un-checkpointed WAL	recovery_s $\leq$ the RTO budget at the maximum WAL volume permitted by max_wal_size; the relationship is linear and predictable, so the worst case is computable from configuration	P0
NL-C-05	Repeated crash cycles	10 $\times$ (kill -9, restart) at 5 s intervals	recovery_s at cycle 10 $\leq$ 1.5 $\times$ cycle 1; no cumulative bloat	P1

NL-C-06	Crash during CREATE INDEX CONCURRE NTLY	kill -9 mid- build	Index left INVALID, table readable, rebuild succeeds	P1
NL-C-07	Crash while acting as a backup source	Kill the primary while pg_baseback up is streaming	Backup fails cleanly and is detectably incomplete — never silently truncated	P1

10.3 NL-W · WAL and archiving

ID	Scenario	Fault injection	Acceptance criterion	Pri
NL-W-01	Commit durability with markers	Marker- instrumented load, then power loss	local_marker s – db_markers == 0	P0
NL-W-02	WAL segment boundary crash	Power loss during a segment switch	No WAL corruption; recovery completes	P0
NL-W-03	WAL segment corruption in pg_wal	Flip bytes in an unreplayed segment, then restart	Truncation LSN reported and a <i>recovery- ended-early</i> warning raised; transactions lost after the	P0

			corruption point quantified. Never a silent early end to replay.	
NL-W-04	Archive command failure	archive_command returns non-zero for 30 minutes	Zero WAL loss; pg_wal stays $\leq 2 \times \text{max_wal_size}$ plus archive backlog and never exceeds 90% of filesystem capacity; MTTF ≤ 300 s	P1
NL-W-05	Archive under sustained pressure	Throttle the archive to 50% of the WAL generation rate	No gap in the archive sequence; backpressure applied before the disk fills	P1
NL-W-06	Archive gap mid-chain	Delete a segment from the archive, then attempt PITR	Restore fails at the exact gap with the correct LSN reported	P1
NL-W-07	pg_wal filesystem	Fill pg_wal to 100% under	PANIC is clean and	P0

	full	load	recoverable; corruption_c ount == 0; restarts unattended once space is freed	
--	------	------	--	--

10.4 NL-I · Integrity and corruption

ID	Scenario	Fault injection	Acceptance criterion	Pri
NL-I-01	Heap page checksum	Corrupt a byte in a heap file	Detected on read; the error identifies the relation and block	P0
NL-I-02	Index corruption	Corrupt a btree file	amcheck detects it; REINDEX restores; no wrong results returned first	P0
NL-I-03	CLOG / pg_xact corruption	Corrupt a CLOG page	Detected, not silently misinterpreted; the documented recovery path succeeds	P0
NL-I-04	pg_control corruption or	Corrupt or delete	Recovery completes	P0

	loss	pg_control	via the documented runbook with no operator improvisation ; lost transactions counted and reported , not estimated	
NL-I-05	Visibility map / FSM corruption	Corrupt the _vm and _fsm forks <i>separately — their severities are opposite</i>	FSM: self-heals (not WAL-logged, rebuilt); no wrong results. VM: a falsely-set all-visible bit must not cause index-only scans to return dead rows; detected by pg_visibility's pg_check_visible() / pg_check_frozen() — <i>not</i> by amcheck, which does not verify the visibility map — and vacuum must not skip the page	P1

NL-I-06	TOAST corruption	Corrupt a TOAST chunk	Detected on detoast; the error identifies the row	P1
NL-I-07	Full pg_amcheck sweep	pg_amcheck --all --heapallindexed post-recovery	errors == 0 after every Tier-1 and Tier-2 scenario	P0
NL-I-08	Logical invariant validation	Application-level FK / sum / count invariants post-recovery	invariant_violations == 0	P0
NL-I-09	Checksum-disabled detection	Run against a cluster with data_checksums=off	The framework refuses to certify and reports the gap	P1

10.5 NL-R · Resource exhaustion

ID	Scenario	Fault injection	Acceptance criterion	Pri
NL-R-01	OOM kill of a backend	cgroup limit; trigger the OOM killer on a backend	Postmaster survives or restarts; rpo_txn == 0; rto_first_writes ≤ 60	P0
NL-R-02	OOM kill of the	OOM killer targets the	Recovery is unattended;	P0

	postmaster	postmaster	corruption_count == 0	
NL-R-03	Data filesystem full	fallocate to 100% under load	Writes rejected with an explicit error; reads continue; no corruption; recovers when space is freed	P0
NL-R-04	Connection exhaustion	Open max_connections + 50%	Rejections are explicit; superuser reserved slots honoured; existing sessions unaffected	P1
NL-R-05	File descriptor exhaustion	Low ulimit -n, touch many relations	Explicit error; no crash; no corruption	P1
NL-R-06	CPU saturation from our own workload	stress-ng --cpu to 100% for 120 s	No backend crash; checkpoint completes; WAL writes continue; $p99 \leq 5 \times$ baseline and degradation attributed to CPU in telemetry.	P0

			<i>Failover suppression under this condition is asserted by CL-F-07, not here — this scenario stays node-local.</i>	
NL-R-07	I/O saturation	fio saturating the data volume	p99 degrades but there is no crash; no false failover; checkpoint completes	P1
NL-R-08	Temp space exhaustion	Fill temp_tablespace with a large sort	The offending query fails; no backend crash; other sessions sustain $\geq$ 95% baseline TPS	P2
NL-R-09	work_mem memory pressure	Many concurrent hash joins at high work_mem	Spill to disk or explicit failure; no OOM cascade	P1

10.6 NL-N · Noisy neighbour and contention

Why this is a separate category from NL-R

NL-R injects resource pressure **on the target node, under our control**. Noisy neighbour is **external, invisible from inside the guest, and usually partial and sustained** rather than a clean failure. The signatures are entirely different:

- **CPU steal** — the guest sees *low* load and *high* latency at the same time. Nothing inside the database looks busy. Operators and health probes both misread it.
- **cgroup throttling** — bursty stalls at CFS period boundaries, not sustained saturation. Average utilisation can look healthy while p99 collapses.
- **Burst-credit depletion** — a cloud volume performs to specification for hours, then falls off a cliff with no configuration change and no error.

This category also covers **grey failure**: sustained partial degradation that never crosses a threshold, is never alerted, and is the hardest failure class to detect. It is also the most common real-world *trigger* for the spurious failover tested in CL-F-07 — that scenario tests the consequence; these test the cause.

ID	Scenario	Fault injection	Acceptance criterion	Pri
NL-N-01	Hypervisor CPU steal	A competing VM saturating the host; measure guest %st	Steal correctly identified as steal, not misdiagnosed as database load; leader_changed == false; p99 degradation < 3x baseline	P0

NL-N-02	cgroup CPU throttling	systemctl set-property postgresql.service CPUQuota=50% — no container required	cpu.stat throttling counters exposed and alerted (nr_throttled plus throttled_usec on cgroup v2, throttled_time on v1); no false health-probe timeout; leader_changed == false	P0
NL-N-03	Shared storage IOPS exhaustion	A co-tenant saturates shared SAN/EBS; or drive burst-credit depletion to zero	fsync p99 ≤ 10 × baseline and attributed to storage in telemetry; checkpoint completes; WAL stall alerted; rpo_txn == 0; no spurious failover	P0
NL-N-04	Host memory pressure / ballooning	Hypervisor reclaims guest memory; cgroup	No OOM kill of the postmaster; shared_buffers resident set ≥ 50% of	P1

		reclaim without OOM	the configured size; TPS $\geq$ 50% baseline; degradation detected	
NL-N-05	Network bandwidth contention	A co-tenant saturates the shared NIC	replication_lag_s $\leq$ 60 during contention, returning to $\leq$ 1 s within 60 s of it clearing; backpressure applied; leader_changed == false	P1
NL-N-06	Internal — backup contending with peak OLTP	pgBackRest full backup on the same host and volume during peak load	OLTP p99 impact < 20%; backup completes; no failover triggered	P1
NL-N-07	Internal — analytics starving OLTP	A long analytic query with large sorts evicting shared buffers	OLTP TPS $\geq$ 70% baseline and p99 $\leq$ 2 $\times$ baseline; buffer cache hit ratio $\geq$ 80% of baseline; resource limits	P1

			enforced if configured	
NL-N-08	Sustained grey degradation	Reduce effective capacity by 20% for 4 h — no clean failure at any point	Detected and alerted as degradation — neither silence nor spurious failover; degraded-mode SLO met, or degraded state explicitly declared	P0

Cross-tier assertion

The chain *noisy neighbour* → *health-probe timeout* → *spurious failover* → *RPO risk* is the highest-value linkage in the framework. Every NL-N scenario asserts `leader_changed == false` when executed against a clustered target, and NL-N-01/02/03 are the realistic fault sources for CL-F-07.

10.7 NL-M · Maintenance, vacuum and wraparound

ID	Scenario	Fault injection	Acceptance criterion	Pri
NL-M-01	XID wraparound protective shutdown	<code>pg_resetwal -x</code> on a scratch cluster to advance the XID counter	Warnings escalate at documented thresholds; recovery restores	P0

		close to the threshold, then drive a normal workload across it. <i>(Organic wraparound needs ~2.1 bn transactions and is not executable in a test window.)</i>	service; no data loss	
NL-M-02	Multixact wraparound	pg_resetwal -m to advance the multixact counter on a scratch cluster, then generate multixacts via concurrent SELECT ... FOR SHARE across the boundary	Detected and vacuumed; protective shutdown is recoverable	P0
NL-M-03	Autovacuum worker killed	kill -9 an autovacuum worker repeatedly	A new worker spawns; no relation is left permanently unvacuumed	P1

NL-M-04	Autovacuum starvation	Sustained write load exceeding vacuum capacity for 4 h	Dead-tuple ratio $\leq 20\%$; TPS degradation $< 30\%$; alerts fire before the critical threshold	P1
NL-M-05	Idle-in-transaction blocking vacuum	An idle transaction held for 2 h under write load	idle_in_transaction_timeout enforced, or a bloat alert fires	P1
NL-M-06	Rolling minor-version restart	Restart under load with -m fast	Availability gap ≤ 5 s per node; rpo_txn == 0	P1
NL-M-07	Config reload under load	SIGHUP with changed parameters at 1000 TPS	Zero dropped connections; zero failed transactions	P1
NL-M-08	Clock skew and NTP loss	± 5 min offset; stop chronyd	No backend crash; TPS $\geq 95\%$ baseline; skew detected and reported within 300 s	P2

Tier 1 total: 54 scenarios — P0 29 · P1 23 · P2 2

11. Tier 2 — Cluster Scenarios (CL)

Applies to any replicated HA group — the standalone Patroni cluster, the DistDB Primary Coordinator and standby pair, and each DistDB 4-node worker cluster.

Applies key: S = standalone Patroni cluster · PC = DistDB Primary Coordinator plus standby · W = each DistDB worker cluster · all = every node type including Query Coordinators.

11.1 CL-F · Failover and promotion

ID	Scenario	Fault injection	Acceptance criterion	Pri	Applies
CL-F-01	Graceful switchover	Patroni switchover	rpo_txn == 0; rto_first_write_s ≤ 30; rto_to_slo_s ≤ 120	P0	S, PC, W
CL-F-02	Ungraceful primary failure	kill -9 the primary	rpo_txn == 0; rto ≤ 60; full RTO decomposition recorded; Elle reports no G0/G1a/G1b/G1c/G-single/G2 anomalies in the recorded history at the declared	P0	S, PC, W

			isolation level		
CL-F-03	Primary host power loss	VM force-off	rpo_txn == 0; rto ≤ 60; Elle reports no G0/G1a/G1b/G1c/G-single/G2 anomalies in the recorded history at the declared isolation level	P0	S, PC, W
CL-F-04	Cascading failover	Fail the primary, then fail the new primary 60 s later	The second failover succeeds; rpo_txn == 0 both times	P1	S, PC, W
CL-F-05	Failback via pg_rewind	Restart the old primary after failover	Rejoins as a replica automatically; data matches the new primary exactly	P1	S, PC, W
CL-F-06	pg_rewind refusal / timeline	Old primary with	Rewind refuses rather than	P0	S, PC, W

	divergence	diverged WAL after a partition	corrupting; the documented reconciliation succeeds; divergent transactions quantified		
CL-F-07	Failover suppression — transient disturbance	Network drop shorter than ttl; CPU stall; slow disk	leader_changed == false for every sub-threshold disturbance	P0	S, PC, W
CL-F-08	RTO budget decomposition	Standard kill -9, fully instrumented	Each of T_detect / T_elect / T_promote / T_reconnect within its own budget	P0	S, PC, W
CL-F-09	Cold-cache performance cliff	Measure the post-promotion throughput recovery curve	rto_to_slows ≤ 180 at ≥ 80% baseline TPS, p99 ≤ 1.5× baseline	P1	S, PC, W

CL-F-10	Promotion with a lagging replica	Induce 60s of lag, then fail the primary	Either the lagged replica is not promoted, or RPO is measured and reported honestly	PO	S, PC, W
---------	----------------------------------	--	---	-----------	----------

11.2 CL-S · Split-brain and fencing

ID	Scenario	Fault injection	Acceptance criterion	Pri	Applies
CL-S-01	Primary isolated from the consensus store	Block primary → etcd	The primary demotes itself within ttl; dual_write_detected == false; Elle reports no G0/G1a/G1b/G1c/G-single/G2 anomalies in the recorded history at the declared	PO	S, PC, W

			isolation level		
CL-S-02	Full partition with writers on both sides	Symmetric partition, load applied to both sides	Exactly one side accepts writes; the other rejects; Elle reports no G0/G1a/G1b/G1c/G-single/G2 anomalies in the recorded history at the declared isolation level	PO	S, PC, W
CL-S-03	Fencing validation (watchdog / STONITH)	Partial failure, old primary still running. <i>Patroni fences via a watchdog (softdog) plus self-demotion rather than classic</i>	The old primary cannot accept writes once the new primary is promoted	PO	S, PC, W

		<i>STONITH — test whichever mechanism is deployed</i>			
CL-S-04	Consensus store quorum loss	Stop the majority of etcd nodes	The cluster degrades to read-only; no promotion; recovers when quorum returns	P0	S, PC, W
CL-S-05	Consensus store total loss	Stop all etcd nodes	Behaviour is documented; no split-brain; the recovery procedure is validated	P1	S, PC, W
CL-S-06	Asymmetric partition	Primary → replica blocked, replica → primary open	No dual-primary; state converges ; Elle reports no G0/G1a/G1b/G1c/G-single/G2 anomalies	P1	S, PC, W

			in the recorded history at the declared isolation level		
--	--	--	---	--	--

11.3 CL-R · Replication resilience

ID	Scenario	Fault injection	Acceptance criterion	Pri	Applies
CL-R-01	Replica network interruption	Block replication for 5 minutes	Auto-reconnect; no full resync; lag ≤ 1 s within 60 s	P0	S, PC, W
CL-R-02	Replica process crash	kill -9 the replica	Replication resumes on restart; no impact on the primary	P1	S, PC, W
CL-R-03	Sync replica stall	SIGSTOP the sync standby	The primary blocks per configuration; behaviour matches the documented	P0	S, PC, W

			contract exactly		
CL-R-04	Sync quorum degradation matrix	ANY k (...) with k-1, then k-2 standbys available	The availability /durability trade-off at each level matches the documented decision	P0	S, PC, W
CL-R-05	Slot bloat → pg_wal full → PANIC	Disconnect the slot consumer, generate WAL to exhaustion	max_slot_wal_keep_size bounds growth; the slot is invalidated before the disk fills; the alert precedes	P0	S, PC, W
CL-R-06	Cascading replica chain break	Kill the middle node of a cascade	Downstream recovers or fails explicitly; the documented procedure works	P1	S, PC, W

CL-R-07	Continuous consistency validation	Checksum the primary against each replica throughout	divergent_rows == 0 at every sample point	P0	S, PC, W
CL-R-08	Logical slot survival across failover	Fail the primary with active logical subscribers	The slot survives (PG 17 failover slots) or the gap is explicitly documented and mitigated	P0	S, PC, W
CL-R-09	Logical replication conflict	Insert a conflicting row on the subscriber	Conflict detected, logged, resolution policy applied; the apply worker recovers	P1	S, PC, W
CL-R-10	hot_standby_feedback conflict matrix	Long replica query × primary vacuum, feedback on and off	In the configured mode exactly one cost is incurred and is bounded: with	P1	S, PC, W

			feedback off, replica query cancellatio ns occur and the primary dead- tuple ratio stays $\leq$ 20%; with feedback on, zero cancellatio ns and the primary dead- tuple ratio $\leq$ 40%		
--	--	--	---	--	--

11.4 CL-N · Network

ID	Scenario	Fault injection	Acceptance criterion	Pri	Applies
CL-N-01	Latency injection	tc netem delay 500ms	leader_changes == 0; replication_lag_s $\leq$ 30	P1	all
CL-N-02	Packet loss	tc netem loss 10%	No corruption; no spurious failover;	P1	all

			replication continues		
CL-N-03	Bandwidth throttle	Limit the replication link to 10% of the WAL rate	replication_lag_s ≤ 300 while throttled; slot protection engages before pg_wal exceeds max_slot_wal_keep_size	P1	all
CL-N-04	Asymmetric latency	500 ms in one direction only	leader_changes == 0 for the duration	P2	all
CL-N-05	DNS failure	Block resolution	Zero existing connections dropped; new connection attempts fail within connect_timeout with an explicit resolution error, never	P1	all

			hanging indefinitely		
CL-N-06	Intermittent flapping link	10 s up / 10 s down for 5 minutes	No failover storm; leader_changes ≤ 1	P0	S, PC, W

11.5 CL-C · Connection layer

ID	Scenario	Fault injection	Acceptance criterion	Pri	Applies
CL-C-01	Pooler process failure	kill -9 PgBouncer	Recovery ≤ 10 s; the application reconnects; rpo_txn == 0	P1	all
CL-C-02	Pooler behaviour during failover	Failover with the pooler in transaction mode and transactions in flight	In-flight transactions fail explicitly — never silently commit twice or vanish	P0	all
CL-C-03	Reconnect storm	Failover with 5,000 idle clients reconnecting simultaneously	max_connections not exhausted; recovery ≤ 60 s; no thundering	P0	all

			-herd collapse		
CL-C-04	Pooler client-side exhaustion	Exceed max_client_conn	Explicit rejection; no database-side impact	P1	all
CL-C-05	Certificate expiry under load	Expire the server certificate mid-workload	Existing connections behave per the documented contract; new connections fail explicitly; the alert precedes expiry	P2	all

11.6 CL-B · Backup, restore and disaster recovery

ID	Scenario	Fault injection	Acceptance criterion	Pri	Applies
CL-B-01	Full backup integrity	pgBackRest full backup → restore	row_checksum_match == 100%; amcheck_errors == 0	P0	all

CL-B-02	Point-in-time recovery accuracy	DROP TABLE, restore to T-1 s	The exact target LSN is reached; the drop is absent; no over- or under-shoot	P0	all
CL-B-03	Incremental chain restore	Full → incr1 → incr2 → restore	The chain restores correctly; data matches the source exactly	P0	all
CL-B-04	Backup under heavy load	Backup during 2000 TPS	The backup is internally consistent; primary p99 impact < 20%	P1	all
CL-B-05	Restore RTO at production volume	Restore a full-size (≥ 1 TB) backup, timed	restore_s ≤ documented DR RTO; throughput recorded	P0	all
CL-B-06	Backup corruption detection	Corrupt a byte in a backup file	pgbackrest verify detects it before a restore is attempted	P0	all

CL-B-07	Backup repository unavailable	Block object storage during backup, and again during restore	Explicit failure; retry succeeds; no partial backup is marked valid	P1	all
CL-B-08	Cross-region / cross-DC restore	Restore into the DR region	DR RTO and RPO both measured against target	P1	all
CL-B-09	Retention / expiry safety	Run expiry with a restore in flight	No in-use backup is deleted; the chain is never broken	P1	all

11.7 CL-X · Compound failures

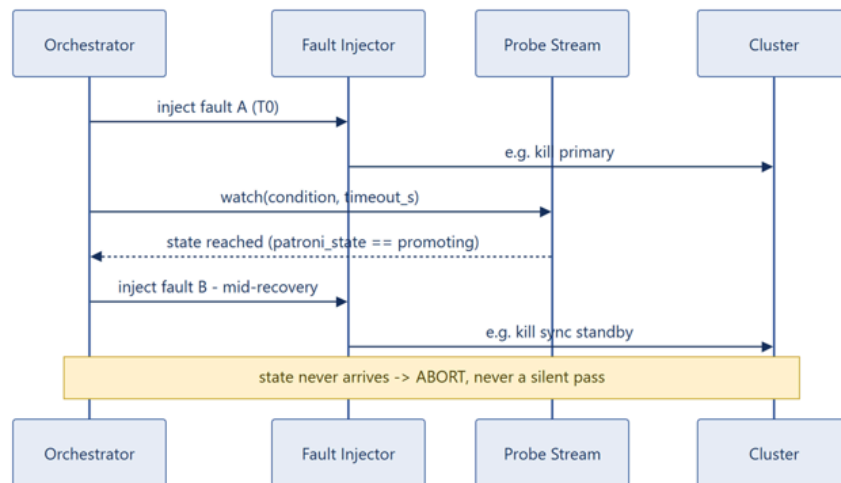
Why this category exists

Every other scenario in this framework is **singular**: steady state → inject one fault → recover → validate. Production incidents are rarely singular. They look like: fault A → the system is now degraded → fault B arrives → *the recovery mechanism itself fails*.

The second fault is not injected at a fixed time. It is injected **at the moment the system is most fragile** — mid-promotion, mid-rebuild, mid-backup — which is precisely the window a singular scenario never visits.

This is a deliberately small, hand-picked suite — not a Cartesian product. Ten pairs chosen because each has a documented history of turning a recoverable incident into an outage. Combinatorial expansion of

155 scenarios would produce tens of thousands of pairs with a vanishing yield per test.



ID	Scenario	Fault A → Fault B (trigger)	Acceptan ce criterion	Pri	Applies
CL-X-01	Sync replica fails during promotion	Kill the primary → kill the sync standby <i>when Patroni enters promoting</i>	Exactly one primary after convergence; rpo_txn == 0, or loss quantified against markers and disclosed; dual_write_detected == false; recovery ≤ 2 × the single-	P0	S, PC, W

			failure RTO budget		
CL-X-02	Consensus store degrades mid-failover	Kill the primary → inject 2 s of etcd latency <i>during the election window</i>	leader_changes ≤ 1; dual_write_detected == false; if the election cannot complete the cluster stays read-only, never split	PO	S, PC, W
CL-X-03	Primary crashes with the data volume near-full	Fill the data filesystem to 95% → kill -9 the primary	corruption_count == 0; recovery completes or fails explicitly — never partial replay; failover proceeds; rpo_txn == 0	PO	S, PC, W
CL-X-04	Primary crashes with the WAL archive	Block the archive destination → kill -9 the primary	rpo_txn == 0; pg_wal_bounded per NL-W-04; the archive	PO	S, PC, W

	unavailable		backlog drains on restore with no gap in the sequence		
CL-X-05	Node restarts while partitioned	Partition a node → restart it <i>while still partitioned</i> → heal	No split-brain at any point; the node rejoins correctly or stays fenced; divergent_rows == 0 after healing	PO	S, PC, W
CL-X-06	Pooler fails during failover	Kill the primary → kill PgBouncer <i>when promotion starts</i>	In-flight transactions fail explicitly — never silently commit twice or vanish; clients reconnect ≤ 60 s after the pooler restarts; rpo_txn == 0	PO	S, PC, W

CL-X-07	Primary fails mid-backup	Start a pgBackRest full backup → kill the primary <i>at ~50% progress</i>	The backup is marked invalid — never partially valid ; failover completes within RTO; the next backup succeeds and pgbackrest verify passes on the resulting chain	PO	S, PC, W
CL-X-08	A second replica fails during a rebuild	Rebuild replica A → kill replica B <i>mid-rebuild</i>	The primary sustains $\geq 95\%$ baseline TPS; the rebuild completes or fails explicitly; sync-quorum behaviour matches the document	PO	S, PC, W

			ed contract (CL-R-04)		
CL-X-09	Primary DC lost while DR replication is lagging	Induce cross-DC lag → stop the entire primary datacentre	DR RPO measured against markers, never assumed; the promotion decision and its data-loss cost recorded; no split- brain across datacentre s. <i>DistDB</i> <i>equivalent:</i> <i>DX-D-03.</i>	PO	S, PC, W
CL-F-10	<i>Primary dies while a replica is lagging</i>	<i>Catalogue d under Failover and Promotion (§11.1). It is the tenth member of this suite and is not duplicated here.</i>			

Two consequences for the harness — neither is optional

1. **Fault B is state-triggered, not time-triggered.** “Kill the sync standby 20 seconds in” is not the same test as “kill it the instant promotion begins” — and only the second reliably hits the fragile window. The orchestrator must arm the second injection on a *predicate over the probe stream*.
2. **Blast radius rises to 2 nodes for this category alone.** Every other scenario is capped at max_nodes_affected: 1. Compound scenarios are, by definition, two concurrent faults. This is a documented, category-scoped exception — **not** a relaxation of the global safety rule — and the Safety Controller must enforce 2 as the hard ceiling here and 1 everywhere else.

Tier 2 total: 55 scenarios — P0 34 · P1 19 · P2 2

12. Tier 3 — Distributed Scenarios (DX)

DistDB only. These scenarios exercise the distributed layer: 2PC (mandatory for DDL, optional for DML), 4-node worker clusters with synchronous replication, 64-bit GXIDs, and a single Primary Coordinator with no consensus protocol for metadata.

12.1 DX-T · Two-phase commit and in-doubt transactions

ID	Scenario	Fault injection	Acceptance criterion	Pri
DX-T-01	Coordinator fails after PREPARE, before COMMIT	Kill the Primary Coordinator between phases	All participants converge to the same outcome; in_doubt_after_recovery == 0; Elle reports no G0/G1a/G1b/G1c/G-	P0

			single/G2 anomalies in the recorded history at the declared isolation level	
DX-T-02	Participant fails after PREPARE	Kill a worker holding a prepared transaction	The prepared transaction survives worker restart; the resolver completes it; Elle reports no G0/G1a/G1b/G1c/G-single/G2 anomalies in the recorded history at the declared isolation level	P0
DX-T-03	Coordinator fails during COMMIT broadcast	Kill the Primary Coordinator mid-broadcast	No participant diverges; a partial commit is resolved, never left split; Elle reports no G0/G1a/G1b/G1c/G-single/G2	P0

			anomalies in the recorded history at the declared isolation level	
DX-T-04	Network partition during PREPARE	Isolate a participant at prepare time	The transaction aborts cleanly on all nodes, or completes on all	P0
DX-T-05	In-doubt resolver under load	1000 concurrent distributed transactions plus a Primary Coordinator kill	All resolved within 120 s; orphaned_prepared_txn == 0	P0
DX-T-06	Prepared transaction lock retention	Leave prepared transactions unresolved for 30 minutes	No unrelated transaction waits > 60 s on a lock held by a prepared transaction; the alert fires within 300 s; no cluster-wide stall	P0
DX-T-07	max_prepared_transactions exhaustion	Exceed the configured limit	Explicit rejection; existing prepared	P1

			transactions unaffected	
DX-T-08	DDL 2PC failure mid-cluster	Kill a worker during distributed DDL	The schema converges cluster-wide, or rolls back cluster-wide — never partial; Elle reports no G0/G1a/G1b/G1c/G-single/G2 anomalies in the recorded history at the declared isolation level	P0
DX-T-09	Multi-shard DML with 2PC disabled	Fail a shard mid-write	Per-shard atomicity honoured; divergence detectable and quantified; Elle reports no G0/G1a/G1b/G1c/G-single/G2 anomalies in the recorded history at the declared isolation level	P0

DX-T-10	GXID exhaustion / wraparound	Pre-seed the GXID counter near the limit on a scratch cluster, then drive distributed transactions across the boundary. <i>Requires a DistDB-side counter-seeding facility — confirm one exists.</i>	Protective behaviour is documented and recoverable	P1
DX-T-11	Distributed deadlock	Cross-shard circular wait	Detected within timeout; one victim aborted; the cluster continues	P1

12.2 DX-M · Metadata and coordinators

ID	Scenario	Fault injection	Acceptance criterion	Pri
DX-M-01	Primary Coordinator crash	kill -9 the Primary Coordinator	DML continues on the Query Coordinators; DDL blocks explicitly; the	P0

			Primary Coordinator recovers within RTO	
DX-M-02	Primary Coordinator failover to standby	Force Primary Coordinator promotion	Metadata intact and complete; Query Coordinators re-sync; metadata_divergence == 0	P0
DX-M-03	Primary Coordinator unavailable for an extended period	Primary Coordinator down for 30 minutes under load	DML TPS $\geq$ 95% baseline; queued DDL bounded by the configured limit with none silently dropped; no cache staleness beyond the documented window	P0
DX-M-04	Metadata cache invalidation lost	Drop invalidation messages to a Query Coordinator	Stale cache detected and corrected; no wrong-shard routing	P0
DX-M-05	Query Coordinator	kill -9 a Query	In-flight queries fail	P0

	crash under load	Coordinator	explicitly; the load balancer routes away; no data impact	
DX-M-06	All Query Coordinators fail simultaneously	Kill every Query Coordinator	The Primary Coordinator absorbs load, or an explicit outage is declared; recovery is unattended	P1
DX-M-07	Query Coordinator added during load	Scale out mid-workload	The new Query Coordinator syncs metadata before serving; no wrong results during warm-up	P1
DX-M-08	Metadata corruption on the Primary Coordinator	Corrupt the catalog on the Primary Coordinator	Detected; the restore-from-backup path is validated	P0
DX-M-09	Split-brain Primary Coordinator	Partition the Primary Coordinator from its standby;	Prevented by fencing; dual_pc_detected == false	P0

		both attempt promotion	
--	--	------------------------	--

12.3 DX-S · Shard placement and rebalancing

ID	Scenario	Fault injection	Acceptance criterion	Pri
DX-S-01	Worker cluster total loss	Kill all 4 nodes of one worker cluster	Affected shards are explicitly unavailable; other shards serve; no wrong results; Elle reports no G0/G1a/G1b/G1c/G-single/G2 anomalies in the recorded history at the declared isolation level	P0
DX-S-02	Shard move interrupted	Kill the source mid-move	The move rolls back or completes; shard_copies == 1; no orphan	P0
DX-S-03	Rebalance under load	Rebalance during 1000 TPS	No lost writes; p99 impact < 30%; placement	P0

			metadata consistent	
DX-S-04	Node addition during a failure	Add a worker while another is down	The addition succeeds or fails cleanly; no metadata corruption	P1
DX-S-05	Node removal with active queries	Drain a worker with queries in flight	Queries complete or fail explicitly; no silent truncation of results	P0
DX-S-06	Reference table replication failure	Fail a worker during a reference-table 2PC write	Reference tables converge on all workers, or roll back everywhere; Elle reports no G0/G1a/G1b/G1c/G-single/G2 anomalies in the recorded history at the declared isolation level	P0
DX-S-07	Colocation constraint under failure	Fail a node holding colocated shards	Colocation is preserved after recovery	P1
DX-S-08	Shard co-tenancy blast	One shard's runaway	The blast radius is	P0

	radius	workload saturates a worker hosting other shards	contained; other shards on that worker stay within SLO or are explicitly declared degraded — never silently slow	
--	--------	--	--	--

12.4 DX-Q · Distributed query execution

ID	Scenario	Fault injection	Acceptance criterion	Pri
DX-Q-01	Worker fails mid-fragment	Kill a worker during a distributed query	The query fails explicitly or retries — never returns partial results as complete; Elle reports no G0/G1a/G1b/G1c/G-single/G2 anomalies in the recorded history at the declared isolation level	P0
DX-Q-02	Straggler worker	SIGSTOP one worker during	Timeout enforced; no	P0

		scatter-gather	indefinite hang	
DX-Q-03	Repartition join with node failure	Fail a worker mid-repartition	Clean failure; temporary and intermediate state cleaned up	P0
DX-Q-04	Result collation failure at the coordinator	OOM the coordinator during aggregation	Explicit failure; no truncated aggregate returned	P0
DX-Q-05	Long analytic query during failover	Failover mid-OLAP query	The query fails explicitly; retry succeeds; no wrong answer	P1
DX-Q-06	Intermediate file cleanup	Kill workers repeatedly during repartition	No unbounded temporary-file accumulation	P1
DX-Q-07	Contended straggler amplification	One worker degraded by a noisy neighbour — slow, not failed (contrast DX-Q-02)	Scatter-gather latency = max(shard); degradation attributed to the correct worker, not to the query; a timeout or	P1

			hedged read bounds the impact	
--	--	--	-------------------------------------	--

12.5 DX-P · Cross-node partition

ID	Scenario	Fault injection	Acceptance criterion	Pri
DX-P-01	Coordinator ↔ worker partition	Block PC/QC → a subset of workers	Affected shards are explicitly unavailable; no stale reads served as fresh; Elle reports no G0/G1a/G1b/ G1c/G- single/G2 anomalies in the recorded history at the declared isolation level	P0
DX-P-02	Worker ↔ worker partition	Block inter- worker traffic	Repartition queries fail explicitly; no wrong results; Elle reports no G0/G1a/G1b/ G1c/G- single/G2 anomalies in the recorded history at the	P0

			declared isolation level	
DX-P-03	PC ↔ QC partition	Block metadata synchronisation	Query Coordinators serve from cache within the staleness bound, then refuse	P0
DX-P-04	Single worker isolated	Isolate one worker node	Its shards fail over within the worker cluster; RTO within target	P0
DX-P-05	Partition healing convergence	Restore all links after 10 minutes	divergent_rows == 0; convergence completes ≤ 300 s after link restore	P0
DX-P-06	Partition exactly at the commit point	Partition timed to the 2PC commit boundary	See DX-T-04; no participant divergence; Elle reports no G0/G1a/G1b/G1c/G-single/G2 anomalies in the recorded history at the declared isolation level	P0

12.6 DX-D · Multi-datacentre disaster recovery

ID	Scenario	Fault injection	Acceptance criterion	Pri
DX-D-01	Primary datacentre total loss	Stop everything in DC1	DR promotion within the DR RTO; RPO measured and within target	P0
DX-D-02	Cross-DC link loss	Sever inter-datacentre replication for 30 minutes	No split-brain; cross-DC lag $\leq$ the documented DR RPO bound; catch-up completes $\leq 2 \times$ the outage duration	P0
DX-D-03	DR promotion with lag	Promote the DR site while lagging	RPO honestly measured and reported — never assumed zero	P0
DX-D-04	Failback to the primary datacentre	Return to DC1 after DR operation	Data reconciled; <code>divergent_rows == 0</code> ; Elle reports no G0/G1a/G1b/G1c/G-	P1

			single/G2 anomalies in the recorded history at the declared isolation level	
DX-D-05	Partial datacentre failure	Lose half of DC1's nodes	The cluster degrades predictably; no data loss	P1
Tier 3 total: 46 scenarios — P0 34 · P1 12				

13. Design Standards

These standards define what counts as a valid resilience test in this programme. Each one changes what must be built, and in what order.

13.1 Durability cannot be validated by process kill

Killing the postmaster terminates the process but leaves the OS page cache fully intact — the kernel still writes those dirty pages to disk afterwards. Such a test exercises PostgreSQL's WAL replay path and **nothing below it**.

Configurations that pass a kill -9 test every time while losing committed data in a real power event: fsync=off · synchronous_commit=off · a volatile write-back controller cache without BBU · a hypervisor in cache=unsafe or writeback mode · volume drivers that ignore fdatasync · incorrectly mounted network storage.

Method	What it proves
pg_test_fsync baseline	Whether flushes are plausibly real — suspiciously high ops/sec indicates a lying cache
VM hard power-off (virsh destroy, cloud force-stop)	Hypervisor and guest cache honesty

dm-flakey / dm-error device-mapper injection	Block-layer write loss and torn writes
Physical power cut on a lab node	Ground truth, including the controller cache

13.2 Acceptance criteria must be falsifiable

Phrases such as “graceful degradation”, “no crash”, “system functional”, “controlled behaviour”, “documented behaviour” and “recovery within SLA” (with no number given) cannot gate a build. Every criterion in this framework is a **numeric or boolean predicate**, and this is enforced mechanically (§13.7, check 3).

13.3 Failover suppression is a first-class requirement

Testing only that failover *happens* is half the requirement. In production, spurious failover triggered by a transient network blip, a CPU starvation stall, a long checkpoint or a slow disk is a leading cause of avoidable outage — and each spurious failover carries real RPO risk. **CL-F-07**, **CL-N-06** and **NL-R-06** assert the negative: that the cluster stays put when it should.

13.4 Noisy neighbour and grey failure are a distinct class

Self-injected resource pressure on the node under test is a fundamentally different failure signature from external contention on shared infrastructure, and it misses the hardest classes entirely:

Mechanism	Why stress-ng does not reproduce it
Hypervisor CPU steal	The guest sees <i>low</i> load with <i>high</i> latency. Nothing inside the database looks busy, so both operators and health probes misdiagnose it.
cgroup CPU throttling	Bursty stalls at CFS period boundaries. Average utilisation looks healthy while p99 collapses.

Burst-credit depletion	The volume performs to specification for hours, then falls off a cliff — no configuration change, no error, no event.
Grey failure	Sustained partial degradation that never crosses a threshold, never alerts, and is never attributed to the right cause.

Category **NL-N** (8 scenarios) plus **DX-S-08** and **DX-Q-07** cover this class. They also supply the realistic *cause* for the contention-induced spurious failover tested in CL-F-07.

13.5 Consistency checking must reach transaction-isolation anomalies

Predicates such as `invariant_violations == 0`, `divergent_rows == 0` and “divergence detectable and quantified” detect **gross divergence only**. None of them can detect a **transaction-isolation anomaly** — lost update, read skew, write skew, aborted read (G1a), intermediate read (G1b), or a dependency cycle (G1c/G2).

For a distributed database in which **2PC is optional for DML**, that would be a material gap. DX-T-09 is precisely the scenario most likely to produce isolation anomalies.

Standard

18 scenarios additionally require Elle — the transactional-anomaly checker from the Jepsen project — to report no G0/G1a/G1b/G1c/G-single/G2 anomalies in the recorded operation history at the declared isolation level. This is an *additional* predicate, not a replacement: the existing timing and integrity criteria still apply.

Scenarios: NL-C-03 · CL-F-02/03 · CL-S-01/02/06 · DX-T-01/02/03/08/09 · DX-P-01/02/06 · DX-Q-01 · DX-S-01/06 · DX-D-04.

Elle is adopted as a **checker over an exported history**, not as a test framework. The harness stays in Python.

13.6 One machine-readable source of truth

The catalog lives in **one YAML/CSV file** from which every document — this one included — is generated. Hand-maintaining parallel Markdown, HTML and CSV copies produces silent

divergence, and divergence in a compliance artefact is a defect.

13.7 Catalog validation — checks applied

Subjective read-throughs surface a different set of defects on every pass, so review never converges. The checks below are **mechanical and repeatable**, and are re-run against the machine-readable catalog on every change.

#	Check	Rule	Last result
1	Row well-formedness	Every scenario row carries ID, name, fault, criterion and priority (plus <i>applies</i> where the table defines it)	155 / 155 pass
2	ID contiguity	Each category numbered 1..n with no gaps or duplicates	20 categories, no gaps
3	Criterion falsifiability	Every acceptance criterion contains a numeric threshold or a boolean predicate; no vague term may be the <i>operative</i> assertion	155 / 155 pass
4	Scenario cross-references	Every referenced scenario ID is defined somewhere in the catalog	No dangling references

5	Section cross-references	Every §n.n reference resolves to an existing heading	All resolve
6	Numeric reconciliation	Tier totals, category counts, priority splits, the coverage matrix and all derived arithmetic agree with the row data	All reconcile

On the thresholds introduced by check 3

Values such as “ $p99 \leq 5 \times \text{baseline}$ ” or “dead-tuple ratio $\leq 20\%$ ” are **initial defaults chosen to make each criterion testable**. They are deliberately expressed relative to measured baseline where possible, so that they port across environment classes. Consistent with open decision 5 (§19), they must be **re-baselined against measurement** before being treated as contractual.

14. Scope Boundaries and Deferred Items

The following are known and consciously deprioritised for this release. They are recorded so that they are not later rediscovered as incidents, and are revisited at each quarterly review.

Deferred item	
Cluster-consistent backup and restore for DistDB (no DX-B category)	CL-B is per-node. Restoring shards to different points yields a silently inconsistent database; cluster-wide PITR across coordinator metadata and N worker clusters is untested.
The HA agent (Patroni) as its own failure domain	If the agent dies while PostgreSQL is healthy, the leader key expires

	and another node promotes while the old primary still accepts writes — with nothing alive to demote it. This is the failure the Patroni watchdog exists to prevent.
Delayed standby (recovery_min_apply_delay)	A deployed protection mechanism against logical corruption, not yet validated.
Monitoring and detection stack failure	Detectability is one of the five properties in §2.2 and currently has no failure scenario. If the exporter dies, MTTD is unbounded.
Distributed rolling upgrade / mixed-version cluster	NL-M-06 covers single-node minor restart only.
Lower-severity items	Prepared transactions across failover on standalone PostgreSQL · lock-table exhaustion · inode exhaustion · shared-memory and semaphore limits blocking restart · auth-backend outage and emergency access · an orphaned slot on a promoted primary · a scale_sensitive axis to validate SLA targets at production data volume.

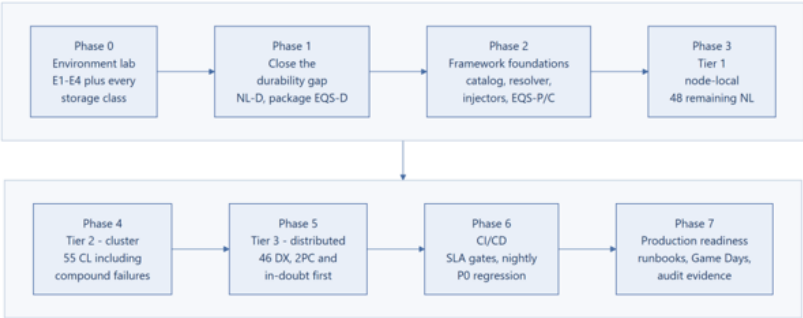


Figure 11 — Delivery phases. Durability is closed before the framework is built.

Phase	Focus	Content	Exit gate
-------	-------	---------	-----------

Phase 0	Environment lab	Stand up a lab representation of every supported class: E1 bare metal, E2 dedicated-host VMs (with a co-tenant VM for NL-N), E3 shared cloud, E4 burstable. Every storage class to be certified is represented alongside.	Every supported environment class is reproducible in the lab
Phase 1	Close the durability gap	NL-D-01 ... NL-D-06 across every certified storage class. Power-loss harness, dm-flakey rig, fsync baselines, synchronous_commit matrix. Package as EQS-D.	The RPO = 0 claim is <i>evidenced</i> per storage class; storage classes that cannot support it are identified
Phase 2	Framework foundations and qualification suite	Machine-readable catalog plus document generation; target resolver with a DistDB role strategy; SSH/systemd/tc/dm-*	One Tier-1 scenario runs end-to-end against all node roles; EQS is runnable against an arbitrary customer environment

		injection adapters ; RTO decomposition instrumentation; transaction-marker library; EQS-P and EQS-C packaged with report generation	
Phase 3	Tier 1 — node-local	The remaining 48 NL scenarios, P0 first, executed against standalone and all DistDB roles. NL-N requires a co-tenant load source — a second VM on the same lab host (E2). It cannot be injected on E3.	All Tier-1 P0 scenarios pass on both systems
Phase 4	Tier 2 — cluster	All 55 CL scenarios, including the compound category, against the standalone Patroni cluster, the Primary Coordinator pair, and the worker clusters	All Tier-2 P0 scenarios pass; the RTO budget is decomposed and within target

Phase 5	Tier 3 — distributed	All 46 DX scenarios; 2PC and in-doubt resolution first	All Tier-3 P0 scenarios pass
Phase 6	CI/CD and continuous validation	Jenkins pipeline with SLA gates; JUnit output; nightly P0 regression; Grafana dashboards	The nightly run is green; regressions are detected automatically
Phase 7	Production readiness	Runbook validation; Game Day exercises; audit evidence package; team training	Sign-off

Sequencing rationale

Phase 1 precedes framework work deliberately. Until the durability gap is closed, every other result is reported against an unverified foundation, and no RPO = 0 statement can be supported. It is the highest-value week of work in the programme.

Phase 2 precedes bulk scenario work because the target resolver is what makes Tier-1 reuse across DistDB node roles possible. Building scenarios first would mean rewriting them.

16. Deliverables, Entry and Exit Criteria

16.1 Deliverables

Deliverable	Description	Audience	Frequency
Test Execution Report	Per-scenario results with all	Engineering	Per run

	measured metrics			
SLA Compliance Report	RTO decomposition, RPO, integrity against targets, both systems	Leadership	Weekly	
Durability Evidence Package	NL-D results per storage class, fsync baselines, power-loss logs	Auditors, Leadership	Quarterly	
Environment Qualification Report	Per deployment: measured EQS-D/P/C results, assigned SLA tier, disclosed limitations	Customer , Field Engineering, Sales	Per deployment; re-run on infrastructure change	
Supported Environment Matrix	Which environment and storage classes are certified for which SLA tier	Product, Sales, Support	Per release	
Trend Analysis	Metrics over time, regression detection	All	Monthly	
Incident Analysis	Root cause for any failed scenario	Engineering	As needed	
Audit Evidence Package	Full compliance documentation with raw artefacts	Auditors	Quarterly	

16.2 Entry criteria

- Test environment provisioned, isolated from production, and healthy
- Target cluster running with replication configured and healthy
- data_checksums enabled — the framework refuses to certify without it (NL-I-09)
- Monitoring, alerting and log aggregation operational
- Backup system operational, with at least one verified restore
- Steady-state baseline captured and stable for the declared duration
- Power-loss injection capability available for NL-D (hypervisor force-off at minimum)
- Co-tenant load source available for NL-N on E2 lab hosts — a second VM we control on the same physical host — plus visibility of %st, nr_throttled and storage burst-credit metrics
- Lab representation of every supported environment class (E1, E2, E3, E4) for the environment-sensitive scenario subset

16.3 Exit criteria

- **All 97 P0 scenarios pass** on their applicable systems
- All P1 scenarios pass, or carry documented and accepted remediation
- **The NL-D category passes** — durability is evidenced, not assumed
- RTO within target *with the decomposition recorded*, and RTO-to-SLO within target
- RPO = 0 validated by transaction markers under power loss, not process kill
- Zero split-brain conditions observed across all CL-S scenarios
- Zero spurious failovers across all suppression scenarios
- pg_amcheck clean after every Tier-1 and Tier-2 scenario
- Test report reviewed and approved by the named signatories

17. Risk Register

Risk	Impact	Likelihood	Mitigation and validating scenarios
Committed data loss from a dishonest storage stack	Critical	Medium	Currently unmitigated and unmeasured. NL-D-01 ... NL-D-06
Committed data loss from misconfiguration	Critical	Low	synchronous_commit=on, fsync=on; NL-D-04, NL-W-01
Split-brain	Critical	Low	Patroni fencing; CL-S-01 ... CL-S-06, DX-M-09
Undetected corruption	Critical	Low	Data checksums, pg_amcheck; NL-I-01 ... NL-I-09
Spurious failover causing a self-inflicted outage	High	Medium	CL-F-07, CL-N-06, NL-R-06; realistic triggers in NL-N-01/02/03
Grey failure — sustained partial degradation never detected	High	High	Currently unmitigated and unmeasured. NL-N-08, NL-N-01 ... NL-N-05

Noisy neighbour degrading a shared-infrastructure node	High	High	NL-N-01 ... NL-N-07; DX-S-08 and DX-Q-07 for shard co-tenancy
Customer deploys on storage that discards flushes	Critical	Medium	EQS-D at go-live; Unqualified tier with written disclosure. Outside our control — only detectable, not preventable.
SLA committed without environment qualification	Critical	Medium	Process control: tier assignment must derive from an EQS report, never from a contract negotiation
Fault-injection tooling assumes containers that do not exist	High	Realised	SSH/systemd/tc/dm-* adapters built first (§7.4); container-native adapters deferred
Failover exceeds RTO	High	Medium	Patroni tuning informed by the CL-F-08 decomposition
Post-failover reconnect storm extends the outage	High	Medium	CL-C-02, CL-C-03

Replication slot bloat fills the primary's disk	High	Medium	max_slot_wal_keep_size; CL-R-05
Backup restore fails or exceeds DR RTO	High	Low	CL-B-01 ... CL-B-09, especially CL-B-05 at production volume
2PC in-doubt transactions block the cluster	Critical	Medium	DX-T-01 ... DX-T-06
Distributed query returns partial results as complete	Critical	Low	DX-Q-01, DX-Q-04, DX-P-01
XID or multixact wraparound outage	Critical	Low	NL-M-01, NL-M-02, NL-M-04

Testing risk

All resilience tests execute in isolated, non-production environments. Every scenario declares a blast radius and abort conditions. Production chaos testing requires explicit approval and uses canary patterns with a limited blast radius. The Safety Controller enforces a kill switch, a circuit breaker, and a maximum-affected-nodes cap on every run.

18. Constraints Already Settled

These decisions are made and are treated as fixed inputs to the framework. They are recorded because each one visibly shapes the design.

Constraint	What it drives
Both shared and dedicated infrastructure must be supported	§3.4 environment classes; §3.5 the conditional support contract; §3.6

— the customer chooses.	the Environment Qualification Suite; §6.1 the tiered SLA. Noisy neighbor is a routine production condition, not an edge case.
The database is not containerized.	§7.4 the non-containerized injection toolchain: chaosd and direct OS tooling first, container-native injectors deferred behind the same adapter interface. Environment class E5 is out of scope until containerization exists.
DistDB implements two-phase commit — mandatory for DDL and metadata, optional for DML.	The whole of Tier 3, and in particular DX-T. Any scenario premised on “partial commit without 2PC” would be invalid for this system.

19. Open Decisions Required

These cannot be resolved from existing documentation and materially affect the design. Each requires an owner and a decision before the corresponding phase begins.

#	Decision	Why it matters	Blocks
1	Is RPO = 0 actually required, and is the availability cost accepted?	RPO = 0 requires synchronous_commit = on or remote_apply with a healthy sync standby — which means the primary blocks when sync quorum is lost. That is a deliberate availability trade-	Phase 1

		off that has never been explicitly accepted in writing. NL-D-04 and CL-R-04 exist to force this into the open.	
2	Is a lab node available where physical power can be cut?	If not, NL-D-02 and NL-D-06 rely on hypervisor force-off — weaker evidence. Acceptable, but the limitation must be stated in any compliance claim.	Phase 1
3	What is the real production data volume?	CL-B-05 (restore RTO) is meaningless at 10 GB if production is 5 TB. DR RTO cannot be validated without this figure.	Phase 4
4	Are the current SLA targets derived or aspirational?	$RTO \leq 60$ s, $RPO \leq 0$ and failover ≤ 30 s appear to be stated rather than measured. The recommendation is to measure	Phase 3

		first, then commit.	
5	Which environment classes are contractually supported at launch?	E1–E4 is the assumed answer, but each class we support requires lab representation and multiplies the 21 environment-sensitive scenarios. Dropping E4 (burstable) would be defensible — credit cliffs make any RTO guarantee fragile — and would reduce both lab cost and support risk.	Phase 1
6	Which storage classes will we certify for RPO = 0?	Durability is a property of storage, not compute. Each certified class (local NVMe, BBU RAID, SAN, network block, NFS) needs its own EQS-D run and lab representation.	Phase 1

		NFS in particular is high-risk and may warrant explicit exclusion.	
7	Is the Environment Qualification Suite a shipped product deliverable or an internal/field-engineer tool?	If customer-runnable it needs packaging, documentation, safety rails (EQS-D destroys the instance under test) and a defensible report format. If field-only, it can stay internal. This materially changes Phase 2 scope.	Phase 2
8	Is containerization on the roadmap, and when?	Determines whether container-native fault-injection adapters should be built now or later, and whether E5 scenarios should be authored now or later.	Phase 2

20. Document Approval

Role	Name	Signature	Date
------	------	-----------	------

Chief Architect			
DistDB Engineering Lead			
Product Owner			
QA Lead			
Engineering Director			

Unified Resilience Testing Framework — SakthiDB (PostgreSQL) and DistDB

Version 1.0 · Database Reliability Engineering · CONFIDENTIAL — INTERNAL USE ONLY

155 scenarios · 97 PO · 3 tiers · 2 systems

Review quarterly, or after any significant infrastructure or architecture change.